Chương 2. Các mô hình dữ liệu

Mô hình dữ liệu quan hệ (Phần 2): SQL, đại số quan hệ và ràng buộc

Đỗ Thanh Hà

Khoa Trí tuệ nhân tạo
Học viện Công nghệ Bưu chính Viễn thông

Ngày 2 tháng 10 năm 2026

Nội dung

- **Khóa của quan hệ** - nhắc lại và làm rõ
- **Dùng SQL để định nghĩa lược đồ quan hệ** (minh họa trên MySQL)
  - Các loại đối tượng quan hệ, kiểu dữ liệu, lưu trữ vật lý
  - `CREATE` / `ALTER` / `DROP`; `NULL`, `DEFAULT`, `PRIMARY KEY`, `UNIQUE`, tự tăng
- **Đại số quan hệ** - ngôn ngữ truy vấn hình thức
- **Biểu diễn ràng buộc bằng đại số quan hệ**

Mục tiêu

Sau bài học, sinh viên có thể: (1) viết được câu lệnh DDL đầy đủ ràng buộc cho một lược đồ quan hệ; (2) đọc, viết và tính được kết quả của một biểu thức đại số quan hệ; (3) diễn đạt một ràng buộc nghiệp vụ dưới dạng biểu thức đại số quan hệ bằng $\emptyset$ hoặc $\subseteq$.

Khóa của quan hệ - nhắc lại

- Tập thuộc tính K là **khóa** của quan hệ R nếu *không có hai bộ nào* trùng nhau trên K - điều này phải đúng ở **mọi thể hiện hợp lệ**, không chỉ ở thể hiện đang có.
- Trên lược đồ, thuộc tính khóa được **gạch chân**:

Movies($\underline{\text{title}}$, $\underline{\text{year}}$, length, genre)

title	**year**	length	genre
Inception	2010	148	Sci-Fi
Inception	2025	150	Sci-Fi
Titanic	1997	195	Romance

⇒ `title` một mình **không đủ** làm khóa (hai phim cùng tên, khác năm).

⇒ Cặp `(title, year)` mới bảo đảm **duy nhất**.

Khóa là ràng buộc trên lược đồ, không phải quan sát trên dữ liệu

Nếu thể hiện hiện tại tình cờ không có `genre` trùng nhau, ta *không* được kết luận `genre` là khóa. Khóa do người thiết kế **khẳng định về nghiệp vụ** và hệ quản trị CSDL sẽ cưỡng chế nó về sau.

Ví dụ: CSDL ngân hàng - phát hiện lỗi thiết kế

1. Quan hệ `Accounts`

acctNo	type	balance
12345	savings	12000
23456	checking	1000
34567	savings	25

2. Quan hệ `Customers` — **thiết kế SAI**

firstName	lastName	idNo	acctNo
Robbie	Banks	901-222	12345
Lena	Hand	805-333	12345
Lena	Hand	805-333	23456

Hãy tìm mâu thuẫn

`idNo` được khai là khóa chính, nhưng giá trị 805–333 xuất hiện **hai lần**. Vì sao lại như vậy, và phải sửa lược đồ thế nào?

Câu hỏi thảo luận

a) Viết lược đồ quan hệ cho `Accounts`, `Customers` và cho toàn bộ CSDL.

b) Liệt kê các thuộc tính, xác định vai trò của chúng (khóa chính, khóa ngoại).

c) Đề xuất kiểu dữ liệu và miền giá trị phù hợp cho từng thuộc tính (ví dụ: `balance` nên dùng kiểu gì?).

d) Liệt kê các bộ hiện có trong mỗi quan hệ.

e) Trích xuất các thành phần của bộ thứ nhất trong quan hệ `Customers`.

f) Liên kết giữa hai bảng thể hiện qua thuộc tính nào? Điều gì xảy ra nếu xóa tài khoản 12345 trong `Accounts`?

g) **Lược đồ `Customers` ở trên có lỗi. Hãy chỉ ra và sửa lại.**

Gợi ý lời giải (a, b, d, e)

a) Lược đồ quan hệ

- *Accounts*(*acctNo*, *type*, *balance*)
- *Customers*(*idNo*, *firstName*, *lastName*)
- *Owns*(*idNo*, *acctNo*) - xem phần sửa lỗi ở slide sau
- Lược đồ CSDL: $\mathcal{S} = \{Accounts, Customers, Owns\}$

b) Thuộc tính và khóa

- `Accounts`: khóa chính `acctNo`.
- `Customers`: khóa chính `idNo`.
- `Owns`: khóa chính `(idNo, acctNo)`; *cả hai* đồng thời là khóa ngoại.

d) Các bộ $t_{A1} = (12345, \text{savings}, 12000)$, $t_{C1} = (\text{901-222}, \text{Robbie}, \text{Banks}), \ldots$
e) Thành phần của một bộ $t_{C1}[firstName] = \text{Robbie}$; $t_{C1}[idNo] = \text{901-222}$.
Một quan hệ là một tập hợp các bộ: $r = \{t_1, t_2, \ldots, t_n\}$.

Gợi ý lời giải (g): lỗi và cách sửa

Lỗi

Lược đồ cũ *Customers*(*firstName*, *lastName*, *idNo*, *acctNo*) ngầm định *mỗi khách hàng chỉ có một tài khoản*. Nhưng nghiệp vụ cho phép một khách hàng có nhiều tài khoản (và một tài khoản có thể đồng sở hữu). Đó là liên kết $M{:}N$ - không thể biểu diễn bằng một khóa ngoại. Hệ quả: dữ liệu mẫu *vi phạm chính khóa chính* đã khai báo.

Cách sửa — tách bảng nối (đúng quy tắc $M{:}N$ của bài trước)

`Customers(idNo, firstName, lastName)`
`Accounts(acctNo, type, balance)`
`Owns(idNo, acctNo)` với *idNo* → `Customers`, *acctNo* → `Accounts`

[Lưu ý] Nếu nghiệp vụ thực sự chỉ cho phép *một* tài khoản cho mỗi khách hàng (liên kết $1{:}N$), thì giữ *acctNo* trong `Customers` là đúng - nhưng khi đó dữ liệu mẫu ở trên là dữ liệu *không hợp lệ* và phải bị từ chối khi chèn.

Gợi ý lời giải (c, f): miền giá trị và toàn vẹn

c) Miền giá trị đề xuất

Thuộc tính	Kiểu dữ liệu	Miền giá trị / ràng buộc
`acctNo`	`BIGINT`	Số nguyên dương, duy nhất
`type`	`ENUM/VARCHAR`	{savings, checking} — dùng `CHECK`
`balance`	`DECIMAL(15,2)`	**không dùng** `FLOAT` — sai số làm lệch số dư
`idNo`	`CHAR(7)`	Khuôn dạng `'XXX-XXX'`

f) Toàn vẹn tham chiếu

- Ràng buộc: $\pi_{acctNo}(Owns) \subseteq \pi_{acctNo}(Accounts)$.
- Xóa tài khoản 12345 sẽ để lại *tham chiếu treo* trong `Owns` $\Rightarrow$ hệ quản trị CSDL **từ chối** lệnh xóa (`RESTRICT`), hoặc xóa lan (`CASCADE`) tùy khai báo.

[Lưu ý] Với dữ liệu tài chính, *không bao giờ* dùng `CASCADE` để xóa lịch sử sở hữu tài khoản. Hãy đánh dấu "đã đóng" thay vì xóa vật lý, để giữ khả năng kiểm toán.

Ba loại đối tượng quan hệ trong SQL

- **Bảng cơ sở** (*base table / stored relation*): dữ liệu được lưu trữ vật lý trên bộ nhớ ngoài, tồn tại đến khi bị xóa chủ động; hỗ trợ đầy đủ truy vấn và thay đổi cấu trúc.
- **Khung nhìn** (*view / virtual relation*): "quan hệ ảo", không lưu dữ liệu; được định nghĩa bằng một câu `SELECT` và được tính lại tại thời điểm truy vấn.
- **Bảng tạm** (*temporary table*): tạo bằng `CREATE TEMPORARY TABLE`, chỉ tồn tại trong một phiên kết nối và tự giải phóng khi đóng kết nối; dùng lưu kết quả trung gian.

Liên kết với kiến trúc ba mức (Chương 1)

Bảng cơ sở thuộc *mức khái niệm/mức trong*; khung nhìn chính là hiện thực của *mức ngoài*. Đây cũng là cơ chế tạo **độc lập dữ liệu mức logic**.

[Lưu ý] Khung nhìn là công cụ phân quyền theo hàng/cột: cấp quyền `SELECT` trên view thay vì trên bảng gốc giúp che giấu các cột nhạy cảm (lương, số CMND).

Kiểu dữ liệu chuỗi trong MySQL

Kiểu	Mô tả và đặc điểm
`CHAR(n)`	Chuỗi dài cố định ($0 \leq n \leq 255$); dữ liệu ngắn hơn n sẽ được đệm khoảng trắng bên phải.
`VARCHAR(n)`	Chuỗi dài thay đổi; giới hạn thực tế tới 65.535 byte, chia sẻ cho toàn bộ hàng. Tiết kiệm bộ nhớ hơn `CHAR`.
`TINYTEXT`	Văn bản ngắn, tối đa 255 byte.
`TEXT`	Văn bản dài, tối đa 65.535 byte ($\approx$ 64 KB).
`LONGTEXT`	Văn bản rất lớn, tối đa 4 GB.

- **Unicode:** MySQL không có `NCHAR`/`NVARCHAR` như SQL Server. Để hỗ trợ tiếng Việt và emoji, đặt bảng mã `utf8mb4`.
- **Đơn vị giới hạn:** với `TEXT`, con số 65.535 là *byte* chứ không phải ký tự - một ký tự tiếng Việt trong `utf8mb4` chiếm tới 3 byte, nên số ký tự thực tế nhỏ hơn nhiều.
- **Hiệu năng:** `CHAR` truy xuất nhanh nhưng tốn không gian; `VARCHAR` tiết kiệm hơn nhưng tốn thêm 1–2 byte lưu độ dài.

Kiểu dữ liệu thời gian trong MySQL

Kiểu	Phạm vi và định dạng
`DATE`	Ngày, không có giờ. 1000-01-01 → 9999-12-31. Định dạng `YYYY-MM-DD`.
`DATETIME`	Ngày và giờ. 1000-01-01 00:00:00 → 9999-12-31 23:59:59.
`TIMESTAMP`	Mốc thời gian lưu theo UTC. 1970-01-01 → 2038-01-19 (giới hạn 32-bit). Tự quy đổi theo múi giờ phiên làm việc.
`TIME`	Thời điểm hoặc khoảng thời gian, từ −838:59:59 đến 838:59:59.
`YEAR`	Năm 4 chữ số, từ 1901 đến 2155.

- **Độ phân giải:** thêm tham số *fsp* (0–6) để lưu tới micro-giây, ví dụ `DATETIME(6)`.
- `DATETIME` **hay** `TIMESTAMP`**?** `TIMESTAMP` lưu theo UTC nên phù hợp hệ thống đa múi giờ, nhưng *tràn năm 2038*; `DATETIME` lưu đúng giá trị được nhập, không quy đổi múi giờ.
- **Định dạng nhập liệu:** luôn dùng `'YYYY-MM-DD'` để tránh nhập nhằng ngày/tháng.

Kiểu dữ liệu số trong MySQL

Kiểu	Mô tả và phạm vi (có dấu, mặc định)
`BIT(n)`	Trường bit, $1 \leq n \leq 64$. `BIT(1)` thường dùng cho luận lý.
`TINYINT`	1 byte: $-128 \ldots 127$ (hoặc $0 \ldots 255$ với `UNSIGNED`).
`SMALLINT`	2 byte: $-32.768 \ldots 32.767$.
`INT`	4 byte - kiểu số nguyên phổ biến nhất.
`BIGINT`	8 byte - dùng cho định danh rất lớn.
`DECIMAL(p,s)`	Số thập phân *chính xác* (dấu phẩy tĩnh) - dùng cho tiền tệ.
`FLOAT`	4 byte, dấu phẩy động, độ chính xác đơn.
`DOUBLE`	8 byte, dấu phẩy động, độ chính xác kép.

- **Luận lý:** MySQL không có kiểu `BOOLEAN` riêng; đó chỉ là bí danh của `TINYINT(1)`, với 0 là sai và *mọi giá trị khác* 0 là đúng.
- **Tiền tệ:** luôn dùng `DECIMAL`, *không* dùng `FLOAT/DOUBLE` - dấu phẩy động nhị phân không biểu diễn chính xác được 0,1.
- **Tiết kiệm bộ nhớ:** cột chỉ chứa nhãn lớp 0–10 nên dùng `TINYINT UNSIGNED` thay vì `INT` (tiết kiệm 75% dung lượng).

Lưu trữ vật lý trong MySQL (InnoDB)

MySQL tổ chức lưu trữ qua các **công cụ lưu trữ** (*storage engine*). Với công cụ mặc định **InnoDB**, một CSDL gồm hai thành phần vật lý chính:

- **Tệp không gian bảng** (`.ibd`): chứa dữ liệu thực tế của bảng và các *chỉ mục* (cây B^+). Mỗi bảng nằm trong một tệp riêng nếu bật `innodb_file_per_table`.
- **Hệ thống tệp nhật ký**:
  - **Redo log** (`ib_logfile`): ghi các thay đổi chưa kịp ghi xuống tệp dữ liệu $\rightarrow$ phục hồi sau sự cố (*crash recovery*).
  - **Binary log** (`binlog`): ghi lại mọi lệnh làm thay đổi dữ liệu và cấu trúc $\rightarrow$ sao lưu và nhân bản (*replication*).

Liên kết với Chương 1

Đây chính là **mức trong** của kiến trúc ba mức. Nhờ độc lập dữ liệu vật lý, ta đổi công cụ lưu trữ hay thêm chỉ mục mà không phải sửa một dòng SQL nào của ứng dụng.

[Lưu ý] Binlog chứa toàn bộ dữ liệu đã ghi - kể cả dữ liệu nhạy cảm. Tệp này phải được bảo vệ và mã hóa ngang mức với chính CSDL.

Tạo cơ sở dữ liệu (MySQL)

1. Không gian bảng dùng chung (tùy chọn)

```
CREATE TABLESPACE ts_qlsv
  ADD DATAFILE '/var/lib/mysql-files/qlsv_data.ibd'
  ENGINE = InnoDB;
```

2. Tạo CSDL logic và đặt bảng mã

```
CREATE DATABASE QLSV
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;
```

- `utf8mb4` là bắt buộc để lưu tiếng Việt đầy đủ và emoji.
- Các tham số nhật ký (`innodb_log_file_size...`) được cấu hình tập trung trong `my.cnf`, *không* khai báo khi tạo từng CSDL - khác với SQL Server.

So sánh: MySQL và SQL Server

Thao tác	**MySQL**	**SQL Server (T-SQL)**
Kích thước tệp CSDL	Cấu hình trong `my.cnf`	Khai trực tiếp trong `CREATE DATABASE`
Đổi tên CSDL	**không hỗ trợ**	`ALTER DATABASE ... MODIFY NAME`
Sửa kiểu của cột	`ALTER TABLE ... MODIFY COLUMN`	`ALTER TABLE ... ALTER COLUMN`
Cột tự tăng	`AUTO_INCREMENT`	`IDENTITY(1,1)`
Chuỗi Unicode	`VARCHAR` + `utf8mb4`	`NVARCHAR`

Bài giảng lấy **MySQL** làm hệ quy chiếu.

Tạo, sửa, xóa bảng

Tạo và xóa

```
USE QLSV;

CREATE TABLE SinhVien (
  MaSV   CHAR(8),
  HoTen  VARCHAR(60),
  NgaySinh DATE
);

DROP TABLE SinhVien;
```

Sửa cấu trúc (MySQL)

```
-- them cot
ALTER TABLE SinhVien
  ADD COLUMN Email VARCHAR(100);

-- doi kieu / rang buoc
ALTER TABLE SinhVien
  MODIFY COLUMN HoTen VARCHAR(80) NOT NULL;

-- doi ten va kieu cot
ALTER TABLE SinhVien
  CHANGE COLUMN NgaySinh NgaySinh DATE;

-- xoa cot
ALTER TABLE SinhVien
  DROP COLUMN Email;
```

Tùy chọn NULL và DEFAULT

Quy tắc gán giá trị rỗng cho cột, khai báo khi tạo hoặc sửa bảng.

NULL

Cột có thể nhận giá trị rỗng (chưa biết / không áp dụng). Giá trị rỗng **không phải** số 0, cũng **không phải** chuỗi trống ''.

NOT NULL

Bắt buộc phải có giá trị - không thể để trống khi thêm dữ liệu mới.

DEFAULT

Giá trị dùng khi lệnh `INSERT` không nêu cột đó. Ví dụ: `TrangThai VARCHAR(10) NOT NULL DEFAULT 'active'`.

Ví dụ: `CustomerID` nên là `NOT NULL` để mỗi khách hàng đều có mã; `TenDem` có thể `NULL` vì không phải ai cũng có tên đệm. **Cảnh báo:** mọi phép so sánh với `NULL` đều cho `UNKNOWN`, kể cả `NULL = NULL`. Phải dùng `IS NULL`.

PRIMARY KEY và UNIQUE

`PRIMARY KEY` định danh **duy nhất** mỗi bản ghi; `UNIQUE` chỉ bảo đảm giá trị trong (các) cột là **không trùng**.

PRIMARY KEY

- Không trùng nhau.
- **Không rỗng** — ngầm định `NOT NULL` (toàn vẹn thực thể).
- Mỗi bảng **chỉ có một**.
- Thường được tạo chỉ mục gom cụm.

UNIQUE

- Không trùng nhau.
- **Cho phép rỗng**.
- Mỗi bảng có thể có **nhiều**.
- Dùng để khai báo các *khóa dự bị* còn lại.

Ví dụ: bảng **SinhVien** đầy đủ ràng buộc

```sql
CREATE TABLE SinhVien (
  MaSV       INT UNSIGNED  AUTO_INCREMENT,    -- MySQL: tu tang
  MaSV_Truong CHAR(10)     NOT NULL,
  HoTen      VARCHAR(60)   NOT NULL,
  NgaySinh   DATE,
  Email      VARCHAR(100),
  GPA        DECIMAL(3,2)  CHECK (GPA BETWEEN 0 AND 4),
  PRIMARY KEY (MaSV),
  UNIQUE KEY uq_masv_truong (MaSV_Truong),
  UNIQUE KEY uq_email (Email)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
```

- `MaSV` (khóa thay thế, tự tăng) bảo đảm không có hai sinh viên trùng định danh nội bộ.
- `MaSV_Truong` là *khóa dự bị* thật của nghiệp vụ → khai `UNIQUE` + `NOT NULL`.
- `Email` `UNIQUE` nhưng cho phép rỗng: mỗi email chỉ thuộc về một sinh viên, song sinh viên chưa có email vẫn nhập được.

PRIMARY KEY và UNIQUE: cái gì được chấp nhận?

Giả sử MaSV_Truong là PRIMARY KEY, Email là UNIQUE (cho phép rỗng).

#	MaSV_Truong	Email	Kết quả khi INSERT
1	B21DCAT001	an@ptit.edu.vn	**OK**
2	B21DCAT002	binh@ptit.edu.vn	**OK**
3	B21DCAT001	cuong@ptit.edu.vn	**LỖI** — trùng khóa chính
4	B21DCAT003	an@ptit.edu.vn	**LỖI** — trùng UNIQUE
5	NULL	dung@ptit.edu.vn	**LỖI** — khóa chính không được rỗng
6	B21DCAT004	NULL	**OK**
7	B21DCAT005	NULL	**OK** — *nhiều* NULL vẫn hợp lệ

Hai dòng cuối là điểm hay bị hiểu sai nhất

Dòng 5 minh họa **toàn vẹn thực thể**: khóa chính ngầm định NOT NULL. Dòng 6–7 cho thấy UNIQUE chấp nhận *nhiều* NULL trong MySQL, PostgreSQL và Oracle — vì NULL không so sánh bằng với NULL nên hệ thống không coi là trùng. (SQL Server chỉ cho *một* NULL.)

Cột tự tăng và ba lệnh DML cơ bản

```sql
-- MySQL
id INT AUTO_INCREMENT PRIMARY KEY

-- SQL Server
id INT IDENTITY(1,1) PRIMARY KEY

INSERT INTO SinhVien (MaSV_Truong, HoTen)
VALUES ('B21DCAT001', N'Nguyen Van A');
```

```sql
UPDATE SinhVien
SET    GPA = 3.6
WHERE  MaSV_Truong = 'B21DCAT001';

DELETE FROM SinhVien
WHERE  MaSV_Truong = 'B21DCAT001';
```

Hai thói quen bắt buộc

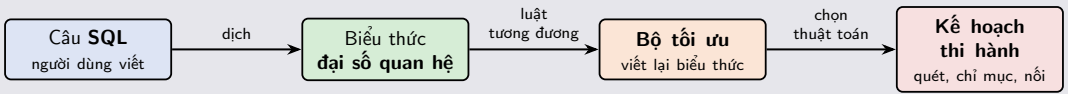
1. `UPDATE`/`DELETE` **luôn** phải có `WHERE` - thiếu nó là tác động lên toàn bảng.
2. Chạy `SELECT` với đúng mệnh đề `WHERE` đó *trước*, xem số dòng trả về, rồi mới đổi thành `UPDATE`/`DELETE`.

[Lưu ý] `CHECK` chỉ thực sự được *cưỡng chế* từ **MySQL 8.0.16** trở đi (và MariaDB 10.2 trở đi). Ở các phiên bản cũ, câu lệnh vẫn chạy nhưng ràng buộc bị *bỏ qua âm thầm* — hãy kiểm tra bằng `SELECT VERSION();` trước khi tin vào nó.

Vì sao cần một ngôn ngữ truy vấn riêng?

- Đại số quan hệ (*relational algebra*, RA) mang lại hai lợi ích lớn:
  - **Dễ diễn đạt**: người dùng mô tả *cái gì* cần lấy, không phải *lấy thế nào*.
  - **Tối ưu hóa được**: vì mỗi biểu thức có *luật biến đổi tương đương*, trình tối ưu của HQT CSDL có thể viết lại biểu thức thành dạng chạy nhanh hơn mà kết quả không đổi.
- RA là một **đại số**: có *toán hạng* và *toán tử*.
  - Toán hạng: biến quan hệ, hoặc quan hệ hằng.
  - Toán tử: các phép ở slide sau — kết quả của mỗi phép *lại là một quan hệ*, nên các phép lồng nhau được (tính **đóng**).

Vị trí của RA trong hệ quản trị CSDL



Học RA là học *cách máy hiểu* câu SQL của mình.

Các phép toán của đại số quan hệ truyền thống

- **Phép toán tập hợp**
  - HỢP — $R \cup S$
  - GIAO — $R \cap S$
  - HIỆU — $R - S$
- **Lấy một phần của quan hệ**
  - CHỌN — $\sigma_C(R)$
  - CHIẾU — $\pi_L(R)$
- **Ghép các bộ từ hai quan hệ**
  - TÍCH DESCARTES — $R \times S$
  - KẾT — $R \bowtie S$, $R \bowtie_\theta S$
- **Đổi tên** — $\rho_{S(A_1,\ldots,A_n)}(R)$

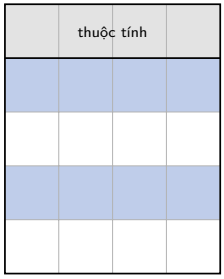
Năm phép toán cơ sở

Chỉ cần $\{\sigma,\ \pi,\ \times,\ \cup,\ -\}$ là đủ: mọi phép còn lại đều biểu diễn được qua năm phép này. Ví dụ $R \cap S = R - (R - S)$ và $R \bowtie_\theta S = \sigma_\theta(R \times S)$.

Mở rộng (không thuộc RA truyền thống): khử trùng δ, gom nhóm và hàm gộp γ, sắp xếp τ, kết ngoài (*outer join*), phép chia $\div$ — cần cho các truy vấn kiểu "giá trị lớn nhất" hay "xuất hiện từ hai lần trở lên".

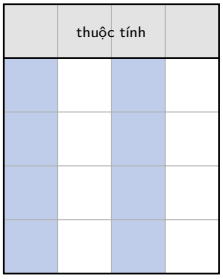
σ cắt theo hàng, π cắt theo cột

$\sigma_C(R)$



giữ các **HÀNG** thỏa điều kiện
lược đồ *không đổi*

$\pi_L(R)$



giữ các **CỘT** được nêu
rồi *khử bộ trùng*

Ghi nhớ: σ *không bao giờ* làm đổi số cột; π *có thể* làm giảm số hàng (do khử trùng) chứ không chỉ giảm số cột.

Điều kiện khả hợp (union-compatibility)

Điều kiện bắt buộc cho $\cup$, $\cap$, $-$

Hai quan hệ R và S chỉ thực hiện được các phép tập hợp khi chúng **khả hợp**:

1. Cùng **bậc** - số thuộc tính bằng nhau.
2. Các thuộc tính tương ứng có **miền giá trị tương thích**.

Nếu tên thuộc tính khác nhau, phải dùng ρ để đổi tên trước.

- $Sells(bar, beer, price) \cup Bars(bar, addr)$ - **không hợp lệ**: bậc 3 và bậc 2.
- $SinhVienCNTT(MaSV, HoTen) \cup SinhVienATTT(MaSV, HoTen)$ - hợp lệ.

Trong SQL

`UNION` mặc định *khử trùng* (đúng ngữ nghĩa tập hợp); `UNION ALL` *giữ nguyên* các bộ trùng (ngữ nghĩa đa tập, chạy nhanh hơn vì không phải sắp xếp/băm để khử trùng).

Phép hợp (Union)

Sells1

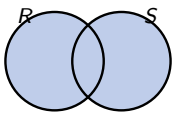
bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50

Sells2

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Miller	3.00

Sells1 ∪ Sells2

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Miller	3.00



Kết quả chứa mọi bộ có mặt ở *ít nhất một* trong hai quan hệ, **đã khử trùng** - vì quan hệ là một tập hợp.

Phép hiệu (Difference / EXCEPT)

Sells1

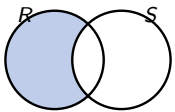
bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50

Sells2

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Miller	3.00

Sells1 − Sells2

bar	beer	price
Sue's	Bud	2.50



Không giao hoán: $R - S \neq S - R$. Ở đây $Sells2 - Sells1 = \{(\text{Sue's}, \text{Miller}, 3.00)\}$. Đây là phép duy nhất trong ba phép tập hợp có tính chất này.

Phép giao (Intersection)

Sells1

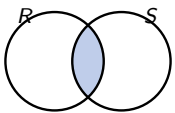
bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50

Sells2

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Miller	3.00

Sells1 ∩ Sells2

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75



∩ *không* phải phép cơ sở: $R \cap S = R - (R - S)$.

Phép chiếu (Projection)

- $R_1 := \pi_L(R_2)$, trong đó L là danh sách thuộc tính thuộc lược đồ của R_2.
- Với mỗi bộ của R_2, giữ lại các thành phần nằm trong L (theo thứ tự đã nêu) để tạo một bộ của R_1.
- **Khử các bộ trùng nhau** sau khi cắt bớt cột.

Sells

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Coors	3.00

Prices $:= \pi_{beer,\ price}(Sells)$

beer	price
Bud	2.50
Miller	2.75
Coors	3.00

Chú ý: `(Bud, 2.50)` xuất hiện hai lần trong `Sells` (ở Joe's và Sue's) nhưng chỉ còn *một* dòng sau phép chiếu. Trong SQL, `SELECT beer, price FROM Sells` *giữ* cả hai; muốn giống RA phải viết `SELECT DISTINCT`.

Phép chiếu mở rộng (Extended Projection)

- Vẫn là toán tử π_L, nhưng cho phép L chứa *biểu thức* trên các thuộc tính:
  1. Tính toán số học và đổi tên kết quả, ví dụ $A + B \to C$.
  2. Lặp lại cùng một thuộc tính nhiều lần (sẽ được tự đánh số A_1, A_2).

R

A	B
1	2
3	4

$\pi_{A+B \to C,\ A,\ A}(R)$

C	A1	A2
3	1	1
7	3	3

Tương ứng SQL: `SELECT A + B AS C, A AS A1, A AS A2 FROM R;`

Phép chọn (Selection)

- $R_1 := \sigma_C(R_2)$, với C là một điều kiện trên các thuộc tính của R_2.
- R_1 gồm *tất cả* các bộ của R_2 thỏa mãn C; lược đồ giữ nguyên.

Sells

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Coors	3.00

JoeMenu $:= \sigma_{bar = \text{'Joe''s'}}(Sells)$

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75

Ký hiệu: phép chọn dùng chữ **sigma** σ. Chữ **delta** δ trong ký pháp Garcia-Molina/Ullman mang nghĩa *khử trùng lặp*, hoàn toàn khác — đừng dùng nhầm.

Cú pháp: dấu nháy đơn trong hằng chuỗi được nhân đôi: `'Joe''s'`.

Tích Descartes (Cartesian Product)

- $R_3 := R_1 \times R_2$: ghép *mỗi* bộ t_1 của R_1 với *mỗi* bộ t_2 của R_2; bộ nối $t_1 t_2$ là một bộ của R_3.
- Lược đồ của R_3 là các thuộc tính của R_1 rồi tới của R_2. Nếu trùng tên thì viết $R_1.A$, $R_2.A$.
- Lực lượng: $|R_3| = |R_1| \times |R_2|$.

$R_1(A, B)$

A	**B**
1	2
3	4

$R_2(B, C)$

B	**C**
5	6
7	8
9	10

$R_3(A,\ R_1.B,\ R_2.B,\ C)$

A	R_1**.B**	R_2**.B**	**C**
1	2	5	6
1	2	7	8
1	2	9	10
3	4	5	6
3	4	7	8
3	4	9	10

[Lưu ý] Tích Descartes hai bảng mỗi bảng 10^5 bộ cho 10^{10} bộ. Trong SQL, quên mệnh đề nối ở `WHERE` chính là vô tình viết một tích Descartes.

Phép kết tự nhiên (Natural Join)

- $R_3 := R_1 \bowtie R_2$ ghép hai quan hệ bằng cách:
  1. Chỉ giữ các cặp bộ có giá trị **bằng nhau trên mọi thuộc tính trùng tên**;
  2. Trong kết quả, mỗi thuộc tính trùng tên chỉ xuất hiện **một lần**.
- Tương đương: $R_1 \bowtie R_2 = \pi_L\big(\sigma_{R_1.A=R_2.A}(R_1 \times R_2)\big)$.

Sells(bar, beer, price)

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Coors	3.00

Bars(bar, addr)

bar	addr
Joe's	Maple St.
Sue's	River Rd.

BarInfo := Sells ⋈ Bars

bar	beer	price	addr
Joe's	Bud	2.50	Maple St.
Joe's	Miller	2.75	Maple St.
Sue's	Bud	2.50	River Rd.
Sue's	Coors	3.00	River Rd.

Kết tự nhiên - hai cạm bẫy

1. Không có thuộc tính trùng tên

Khi đó điều kiện "bằng nhau trên mọi thuộc tính chung" luôn đúng, và $R \bowtie S$ suy biến thành **tích Descartes** $R \times S$. Vì vậy kết tự nhiên rất nhạy cảm với cách *đặt tên cột*: nếu bảng `Bars` dùng tên cột `name` thay vì `bar`, phải ρ lại trước khi kết.

2. Mất bộ "treo" (dangling tuples)

Bộ nào không tìm được bộ khớp ở phía kia sẽ **biến mất** khỏi kết quả. Nếu có một quán bar chưa bán loại bia nào, nó sẽ không xuất hiện trong `BarInfo`.

$\Rightarrow$ Muốn giữ lại, phải dùng **kết ngoài** (*outer join*): trái, phải, đầy đủ - các ô thiếu được điền `NULL`.

Trong SQL: `NATURAL JOIN` tồn tại nhưng *nên tránh* trong mã sản phẩm - chỉ cần ai đó thêm một cột trùng tên là ngữ nghĩa truy vấn thay đổi âm thầm. Hãy viết `JOIN ... ON` tường minh.

Phép kết theta (Theta Join)

- $R \bowtie_\theta S$: gồm mọi cặp bộ của R và S thỏa mãn điều kiện θ.
- Khác kết tự nhiên: điều kiện *tùy ý* (không chỉ dấu bằng), và **giữ nguyên tất cả các cột** của cả hai phía.
- Định nghĩa qua phép cơ sở: $R \bowtie_\theta S = \sigma_\theta(R \times S)$.

R

A	B
1	1
1	2
2	3

S

C	D
2	2
3	2
4	1

$R \bowtie_{B \geq C} S$

A	B	C	D
1	2	2	2
2	3	2	2
2	3	3	2

Kiểm tra: B=1 không có $C \leq 1$; B=2 khớp C=2; B=3 khớp C=2 và C=3. Tổng 3 bộ. Khi θ chỉ gồm các dấu bằng, ta gọi riêng là *kết bằng* (*equi-join*).

Phép đổi tên (Renaming)

- ρ gán một lược đồ mới cho một quan hệ: $R_1 := \rho_{R_1(A_1,\ldots,A_n)}(R_2)$ cho ra quan hệ tên R_1, thuộc tính $A_1, \ldots, A_n$, các bộ giữ nguyên như R_2.
- Viết gọn: $R_1(A_1, \ldots, A_n) := R_2$.
- **Vai trò thiết yếu:** (1) làm cho hai quan hệ trở nên khả hợp; (2) tạo *hai bản sao* của cùng một quan hệ để tự kết (*self-join*) - kỹ thuật bắt buộc khi so sánh các bộ trong cùng một bảng.

Bars(name, addr)

name	addr
Joe's	Maple St.
Sue's	River Rd.

$R(bar, addr) :=$ **Bars**

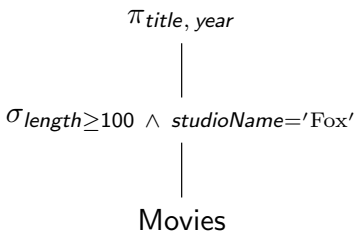
bar	addr
Joe's	Maple St.
Sue's	River Rd.

Tương ứng SQL: mệnh đề `AS` - `FROM Bars AS B1, Bars AS B2`.

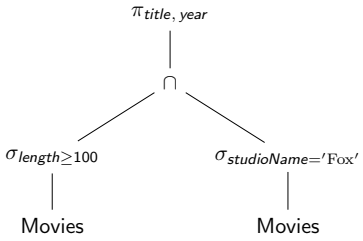
Kết hợp các toán tử: cây biểu thức

`Movies(title, year, length, genre, studioName)`

Truy vấn: tiêu đề và năm của các phim do Fox sản xuất, dài ít nhất 100 phút.



Cách 1 — một phép chọn



Cách 2 — giao của hai phép chọn

Hai cây cho *cùng* kết quả. Cách 1 quét bảng một lần; cách 2 quét hai lần rồi giao. Chính vì tồn tại nhiều cây tương đương như vậy mà HQT CSDL mới cần **bộ tối ưu truy vấn**.

Quan hệ giữa các toán tử

- $R \cap S = R - (R - S)$
- $R \bowtie_\theta S = \sigma_\theta(R \times S)$
- $R \bowtie S = \pi_L(\sigma_{\text{các cột chung bằng nhau}}(R \times S))$

Một vài luật biến đổi mà bộ tối ưu hay dùng

- **Đẩy phép chọn xuống sâu**: $\sigma_C(R \bowtie S) = \sigma_C(R) \bowtie S$ khi C chỉ nhắc tới thuộc tính của R — lọc sớm thì phép kết xử lý ít bộ hơn hẳn.
- **Đẩy phép chiếu xuống sâu**: bỏ sớm các cột không cần.
- **Kết hợp và giao hoán phép kết**: đổi thứ tự kết để bảng nhỏ được xử lý trước.

Đây chính là lý do một câu SQL viết "ngây thơ" vẫn có thể chạy nhanh - HQT CSDL đã viết lại nó giúp bạn.

Đối chiếu đại số quan hệ và SQL

Phép toán	Ký hiệu	Thành phần tương ứng trong SQL
Chọn	$\sigma_C(R)$	`WHERE C`
Chiếu	$\pi_L(R)$	`SELECT DISTINCT L`
Tích Descartes	$R \times S$	`FROM R, S` (không có điều kiện nối)
Kết theta	$R \bowtie_\theta S$	`FROM R JOIN S ON` θ
Kết tự nhiên	$R \bowtie S$	`FROM R NATURAL JOIN S`
Hợp / Giao / Hiệu	$\cup \ \cap \ -$	`UNION` / `INTERSECT` / `EXCEPT`
Đổi tên	$\rho_{S(\dots)}(R)$	`AS`
Khử trùng	$\delta(R)$	`DISTINCT`
Gom nhóm, hàm gộp	$\gamma_L(R)$	`GROUP BY` + `HAVING`
Sắp xếp	$\tau_L(R)$	`ORDER BY`

Khác biệt cần nhớ

`SELECT` của SQL *không* khử trùng, còn π thì có. Vì vậy $\pi_L(R)$ dịch đúng ra SQL phải là `SELECT DISTINCT`, chứ không phải `SELECT`. Nguyên nhân: SQL làm việc trên *đa tập*, còn đại số quan hệ làm việc trên *tập hợp*.

Bài tập đại số quan hệ - lược đồ

- CSDL gồm bốn quan hệ:
  - `Product(maker, model, type)`
  - `PC(model, speed, ram, hd, price)`
  - `Laptop(model, speed, ram, hd, screen, price)`
  - `Printer(model, color, type, price)`
- `Product.model` là khóa ngoại trỏ tới `PC`, `Laptop` hoặc `Printer` tùy giá trị của `type`.

Product

maker	*model*	*type*
A	1001	pc
A	1002	pc
A	1003	pc
A	2004	laptop
A	2005	laptop
A	2006	laptop
B	1004	pc
B	1005	pc
B	1006	pc
B	2007	laptop
C	1007	pc
D	1008	pc
D	1009	pc
D	1010	pc
D	3004	printer
D	3005	printer
E	1011	pc
E	1012	pc
E	1013	pc
E	2001	laptop
E	2002	laptop

PC

model	*speed*	*ram*	*hd*	*price*
1001	2.66	1024	250	2114
1002	2.10	512	250	995
1003	1.42	512	80	478
1004	2.80	1024	250	649
1005	3.20	512	250	630
1006	3.20	1024	320	1049

Laptop

model	*speed*	*ram*	*hd*	*screen*	*price*
2001	2.00	2048	240	20.1	3673
2002	1.73	1024	80	17.0	949
2003	1.80	512	60	15.4	549
2004	2.00	512	60	13.3	1150
2005	2.16	1024	120	17.0	2500

Printer

model	*color*	*type*	*price*
3001	true	ink-jet	99
3002	false	laser	239
3003	true	laser	899
3004	true	ink-jet	120
3005	false	laser	120

Bài tập đại số quan hệ

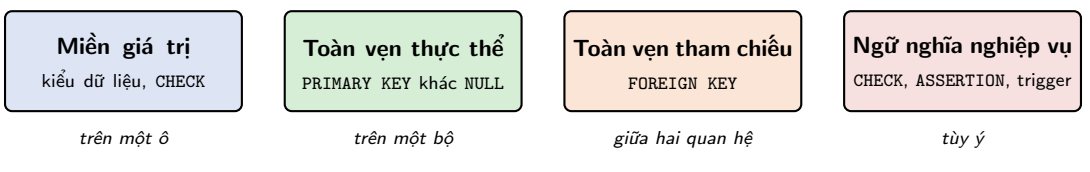
Viết biểu thức đại số quan hệ trả lời các truy vấn sau:

1. Những loại sản phẩm nào do nhà sản xuất A làm ra?
2. Những mẫu PC nào có tốc độ ít nhất 3.00?
3. Những loại sản phẩm nào do *cả* nhà sản xuất A *và* D làm ra?
4. Những loại sản phẩm do A làm ra *nhưng* D không làm?
5. Những nhà sản xuất nào có máy tính xách tay với ổ cứng ít nhất 100 GB?
6. Mã và giá của mọi sản phẩm (thuộc loại bất kỳ) do nhà sản xuất B làm ra.
7. Mã của tất cả máy in laser màu.
8. Những nhà sản xuất làm máy tính xách tay nhưng không làm PC.
9. Những dung lượng ổ cứng xuất hiện ở *từ hai PC trở lên*.
10. Những nhà sản xuất của máy tính có tốc độ cao nhất hiện có.
11. Những nhà sản xuất có ít nhất *hai* máy tính khác nhau (PC hoặc laptop) với tốc độ ít nhất 2.80.

Gợi ý: câu 3, 4, 8 dùng $\cap$ và $-$; câu 9, 10, 11 *không* giải được bằng RA truyền thống nếu chỉ dùng σ, π - phải **tự kết** bằng ρ (hai bản sao của cùng một bảng), hoặc dùng hàm gộp γ.

Ràng buộc - giới thiệu

- Ràng buộc là cơ chế giới hạn các giá trị được phép tồn tại trong CSDL.
- Đã học ở phần trước: ràng buộc *miền giá trị*, *toàn vẹn thực thể*, *toàn vẹn tham chiếu*, và ràng buộc *ngữ nghĩa* do nghiệp vụ đặt ra.
- Phần này trả lời câu hỏi: làm sao **phát biểu chính xác** một ràng buộc, độc lập với mọi cú pháp SQL cụ thể?



Hai dạng biểu diễn ràng buộc bằng đại số quan hệ

- Nếu R là một biểu thức đại số quan hệ thì $R = \emptyset$ là một ràng buộc, nghĩa là: *"kết quả của R phải rỗng"* — không tồn tại bộ nào vi phạm.
- Nếu R và S là hai biểu thức thì $R \subseteq S$ là một ràng buộc, nghĩa là: *"mọi bộ trong kết quả của R cũng phải có trong kết quả của S"* (S được phép chứa thêm bộ khác).

Hai dạng này tương đương

$$R \subseteq S \quad \Longleftrightarrow \quad R - S = \emptyset$$

Ba nhóm ràng buộc sẽ biểu diễn

(1) ràng buộc khóa ngoại; (2) ràng buộc khóa; (3) ràng buộc kiểm tra (*check*).

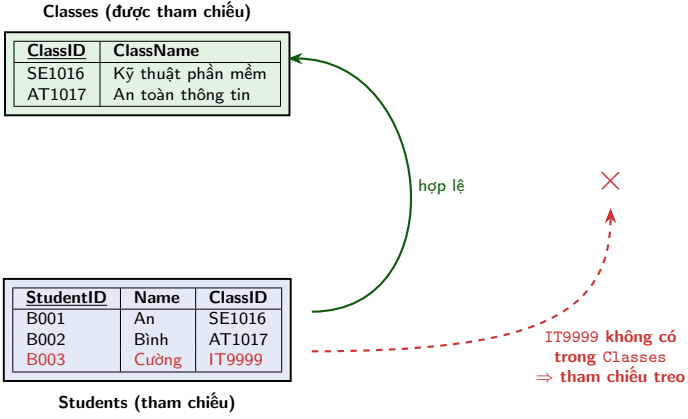
Ràng buộc khóa ngoại

- Ràng buộc khóa ngoại (*toàn vẹn tham chiếu*) khẳng định: một giá trị xuất hiện trong ngữ cảnh này thì cũng phải xuất hiện trong ngữ cảnh liên quan kia.
- Ví dụ: `Classes(ClassID, ClassName, year)` và `Students(StudentID, Name, BirthDate, ClassID)`. Nếu gặp `ClassID = 'SE1016'` trong `Students` thì `'SE1016'` phải có mặt trong `Classes`.
- Tổng quát: R chứa thuộc tính A, S chứa thuộc tính B. Ta nói *"R tham chiếu S qua A và B"* nếu

$$\pi_A(R) \subseteq \pi_B(S) \qquad \text{hay tương đương} \qquad \pi_A(R) - \pi_B(S) = \emptyset$$

[Lưu ý] Nếu A cho phép `NULL`, phải loại các bộ `NULL` trước khi chiếu: $\pi_A\big(\sigma_{A \text{ IS NOT NULL}}(R)\big) \subseteq \pi_B(S)$ — vì `NULL` không vi phạm toàn vẹn tham chiếu.

Toàn vẹn tham chiếu - nhìn bằng hình



Ràng buộc bảo đảm điều này: $\pi_{ClassID}(Students) \subseteq \pi_{ClassID}(Classes)$. Bộ `B003` vi phạm, nên lệnh `INSERT` bị HQT CSDL **từ chối**.

Ví dụ: ràng buộc khóa ngoại

Sells

bar	beer	price
Joe's	Bud	2.50
Joe's	Miller	2.75
Sue's	Bud	2.50
Sue's	Coors	3.00

Bars

bar	addr
Joe's	Maple St.
Sue's	River Rd.

Ràng buộc: mọi quán bar bán bia đều phải có trong `Bars`

$$\pi_{bar}(Sells) - \pi_{bar}(Bars) = \emptyset$$

Kiểm tra: $\pi_{bar}(Sells) = \{\text{Joe's}, \text{Sue's}\}$; $\pi_{bar}(Bars) = \{\text{Joe's}, \text{Sue's}\}$; hiệu bằng $\emptyset \Rightarrow$ dữ liệu hiện tại *thỏa mãn* ràng buộc.

Nếu thêm bộ `(Tom's, Bud, 2.00)` vào `Sells`, hiệu trở thành $\{\text{Tom's}\} \neq \emptyset \Rightarrow$ vi phạm.

Ràng buộc khóa

- Cho lược đồ $R(A_1, A_2, A_3)$. Khẳng định "A_1 là khóa" nghĩa là: *không có hai bộ khác nhau nào trùng giá trị A_1*.
- Lấy hai bản sao $R_1 := \rho_{R_1}(R)$ và $R_2 := \rho_{R_2}(R)$, rồi phát biểu:

$$\sigma_{R_1.A_1=R_2.A_1 \,\wedge\, (R_1.A_2 \neq R_2.A_2 \,\vee\, R_1.A_3 \neq R_2.A_3)}(R_1 \times R_2) \;=\; \emptyset$$

Đọc biểu thức

"Không tồn tại cặp bộ nào cùng giá trị A_1 mà lại khác nhau ở một thuộc tính khác." Đây đúng là kỹ thuật **tự kết** - phải đổi tên bằng ρ thì mới so sánh được hai bộ trong cùng một quan hệ.

[Lưu ý] Dùng σ (sigma - phép chọn). Bản cũ viết δ, là ký hiệu của phép *khử trùng lặp* trong cùng hệ ký pháp - một nhầm lẫn dễ mất điểm khi chấm bài.

Ví dụ: ràng buộc khóa

Quan hệ `Orders`

OrderID	Customer
10248	Vins et alcools Chevalier
10249	Toms Spezialitäten
...	...
10263	Ernst Handel

`OrderID` là khóa

$$\sigma_{R_1.OrderID = R_2.OrderID \ \wedge \ R_1.Customer \neq R_2.Customer}\left(R_1 \times R_2\right) = \emptyset$$

Với quan hệ có n thuộc tính ngoài khóa, điều kiện bất đẳng thức phải nối bằng $\vee$ cho *tất cả* n thuộc tính đó.

Ràng buộc kiểm tra (CHECK)

- Cho lược đồ EMPLOYEE(EmpId, Name, Sex).
- Muốn khẳng định giá trị hợp lệ duy nhất của Sex là 'F' và 'M':

$$\sigma_{Sex \neq 'F' \wedge Sex \neq 'M'}(\text{EMPLOYEE}) = \emptyset$$

Tương ứng trong SQL

```
Sex CHAR(1) NOT NULL CHECK (Sex IN ('F','M'))
```

[Lưu ý] Phải viết *Sex* ≠ 'F' ∧ *Sex* ≠ 'M' (phép *và*), không phải ∨. Với phép *hoặc*, mọi bộ đều thỏa mãn (giá trị 'F' vẫn khác 'M') nên biểu thức không bao giờ rỗng — ràng buộc trở thành vô nghĩa.

Bài tập 1–3: biểu diễn ràng buộc

Bài 1 — PC(Model, Speed, RAM, HDD, Price)

"Một PC có tốc độ dưới 3.00 thì không được bán với giá trên \$800."

Bài 2 — LAPTOP(Model, Speed, RAM, HDD, Screen, Price)

"Một máy tính xách tay có màn hình nhỏ hơn 15.4 inch thì phải có ổ cứng ít nhất 120 GB, hoặc phải được bán dưới \$1000."

Bài 3 — cả PC và LAPTOP

"Nếu một laptop có RAM lớn hơn một PC thì laptop đó cũng phải đắt hơn PC đó."

Gợi ý: với mệnh đề dạng "nếu P thì Q", tình huống vi phạm là $P \wedge \neg Q$. Bài 3 phải so sánh *cặp* bộ từ hai quan hệ khác nhau $\Rightarrow$ dùng $\times$.

Bài tập 4–6: biểu diễn ràng buộc

Bài 4 — PC(Maker, Model), PRINTER(Maker, Model)

"**Không** nhà sản xuất PC nào **đồng thời** sản xuất máy in."

(Đây là trường hợp hai bảng đã có sẵn cột Maker.)

Bài 5 — PRODUCT(Maker, Model), PC(Model, ...), PRINTER(Model, ...)

Cùng ràng buộc như bài 4, nhưng Maker chỉ có trong PRODUCT $\Rightarrow$ phải **kết** để lấy được nhà sản xuất của từng mẫu máy.

Bài 6 — PC(Model, Speed, RAM, HDD, Price)

"Cấu hình cao hơn thì giá phải cao hơn": nếu một PC có *đồng thời* Speed, RAM và HDD lớn hơn hoặc bằng một PC khác, và *ít nhất một* trong ba lớn hơn thực sự, thì giá của nó phải lớn hơn.

Chú ý: bài 4 và bài 5 cùng một ràng buộc nghiệp vụ nhưng khác lược đồ — mục đích là cho thấy *cách lưu trữ quyết định độ phức tạp của biểu thức*. Bài 6 cần tự kết PC với chính nó.

Tổng kết

1. Khóa là **ràng buộc trên lược đồ**, phải đúng với mọi thể hiện — không phải là quan sát rút ra từ dữ liệu đang có.
2. Cú pháp DDL khác nhau giữa các HQT CSDL: `AUTO_INCREMENT` và `MODIFY COLUMN` (MySQL) so với `IDENTITY` và `ALTER COLUMN` (SQL Server). Luôn nêu rõ đang nói về hệ nào.
3. `PRIMARY KEY` ngầm định `NOT NULL` và chỉ có một; `UNIQUE` cho phép `NULL` (nhiều `NULL` theo chuẩn SQL) và có thể có nhiều.
4. Đại số quan hệ có **năm phép cơ sở** $\{\sigma, \pi, \times, \cup, -\}$; mọi phép khác đều dẫn xuất được. Kết quả của mỗi phép lại là một quan hệ (tính đóng).
5. Các phép tập hợp đòi hỏi **khả hợp**; phép chiếu và phép hợp **khử trùng**, còn SQL thì không (phải dùng `DISTINCT`).
6. Ràng buộc được biểu diễn bằng $R = \emptyset$ hoặc $R \subseteq S$; cách làm hiệu quả là mô tả *tình huống vi phạm* rồi đặt bằng $\emptyset$.
7. Muốn so sánh các bộ trong *cùng* một quan hệ, bắt buộc phải đổi tên bằng ρ để tự kết.

Tài liệu tham khảo

- H. Garcia-Molina, J. D. Ullman, J. Widom. *Database Systems: The Complete Book*, 2nd ed., Pearson, 2008 (Ch. 2, 5).
- R. Elmasri, S. B. Navathe. *Fundamentals of Database Systems*, 7th ed., Pearson, 2016 (Ch. 5–8).
- A. Silberschatz, H. F. Korth, S. Sudarshan. *Database System Concepts*, 7th ed., McGraw-Hill, 2020 (Ch. 2, 4).
- Oracle. *MySQL 8.0 Reference Manual* — Data Types, `CREATE TABLE`, `ALTER TABLE`.